电影板块与本地人才结构

基于 1.8 亿条 IMDb 记录的香港发行网络并行图分析

CS5488 Data-Intensive Computing — 小组项目最终报告（中文对照版）

2026 年 11 月

> 本文件是英文正式报告的中文对照版，章节编号、图号、表格数据与英文版完全一致，仅正文语言不同。正式提交以 `Report.docx` 为准。

1. 摘要

香港是全球联系最紧密的电影工业之一：一小群演员和技术人员被数千部作品反复使用，早期围绕片厂制组织，后期围绕独立公司组织。本报告重建了所有在香港上映过的影片的合作网络——工作集为 21,033 部影片、360,502 条署名记录，涉及 IMDb 公开数据集中的 51,878 人，由扫描五个源文件的 1.7 亿行得到——并用三种不同的并行策略加以分析。工程贡献是一套与三种负载形态相匹配的三层设计：针对摄取与连接阶段的字节区间切块 map-reduce；针对 PageRank 的、每轮一次同步的分块稀疏矩阵乘；针对关联规则挖掘的分片无锁计数。在 20 核机器上实测，扫描层在 16 进程达到 6.93× 加速，规则计数在 4.34× 附近饱和，而 PageRank 只达到 1.67×——因为在这个图规模下，一轮迭代只需约一毫秒，该内核受限于内存带宽而非算术量。这三种结果都可以由各负载的内存层次和依赖结构事先推出。在实质性发现上，Louvain 模块度优化无监督地复原了香港发行市场的电影板块：九个社区覆盖全图 68% 的人，包括好莱坞板块、粤语板块、内地板块、邵氏年代板块，以及彼此分离的欧洲和日本板块，模块度 0.75，跨随机种子稳定性 NMI 0.89。在粤语板块内部，PageRank 识别出的是电影史所记载的本地工业核心，而不是作品数量最多的人；各板块的年代画像则勾勒出本地工业在 1990 年代之后的收缩。基于类型与班底项集的关联规则挖掘，在置信度 0.6、提升度 1.2 的门槛之上得到 652 条规则，以动画—冒险的耦合、以及选角与美术指导同冒险类型的共现为主。

关键词： 并行图分析、PageRank、Louvain 社区发现、关联规则挖掘、map-reduce、内存层次、香港电影

---

2. 引言

2.1 背景与问题

香港电影工业在 1950 年代至今生产了数千部剧情片，而在大部分时期里，它依赖的劳动力池异常之小。邵氏与嘉禾各自维持着签约的演员和技术班底；新浪潮以及后来的银河映像则与固定的合作团队反复共事。其结果是：这张合作网络密度高、年代可追溯，且规模小到个体的职业生涯在其中清晰可见——与多数既有研究所关注的、弥散的全球共现网络正好相反。

驱动本项目的分析问题是：这种结构能否被无监督地复原？如果把"谁出现在哪部片里"这份名单交给社区发现算法，它会返回电影史家早已描述的那些板块吗？而基于合作网络计算的中心性，识别出的是工业真正围绕的人，还是仅仅署名列表最长的人？

在数据准备阶段浮现出第二个未曾预料的问题，并最终成为报告的一项发现。IMDb 的公开数据里没有出品国字段。一部影片只能通过其各地区的发行片名被观察到，而"在香港发行"并不等于"香港出品"：每一部在香港开画的好莱坞影片同样拥有香港别名，且常常被译成中文。4.1 节报告了三种工作集定义各自实际筛出了什么，5.4 节则把由此产生的污染当作被测量的威胁、而非假设来处理。对分析而言，其后果是研究对象发生了变化：本报告研究的是**香港发行网络**，本地工业作为其中一个板块出现在网络内部——这一口径事后证明比原来的更有信息量，因为它把"不同电影传统能否被分开"变成了一个可检验的结果。

2.2 目标

O1 — 在规模上重建网络。 摄取完整的 IMDb 公开数据集，限制为有香港发行记录的影片，构建带权、带时间戳的共现图。工程目标：一条分块并行的摄取与连接管线，在单台工作站上扫描 1.7 亿行而不耗尽内存。

O2 — 刻画网络结构。 在共现图上计算度、PageRank 与抽样介数中心性，把"作品多"的人与"把网络撑起来"的人区分开。工程目标：分块稀疏矩阵乘实现，且分块大小由实测行为而非假设决定。

O3 — 复原潜在板块。 应用 Louvain 模块度优化，用标签传播做对照、并跨随机种子验证，再按年代追踪板块成员的变化。工程目标：量化迭代式图算法在何处停止扩展，以及为什么。

O4 — 挖掘类型与班底关联。 把每部影片视为由类型和班底角色构成的购物篮，在不同支持度门槛下挖掘关联规则。工程目标：把候选生成的代价与计数的代价分开度量，并说明两者之中只有一个能并行好。

2.3 数据来源

五个文件全部来自 IMDb 公开数据集，以 gzip 压缩的制表符文本发布在 `https://developer.imdb.com/non-commercial-datasets/`，每日更新。下表数字在本研究所用的发行版（2026 年 11 月 12 日）上测得；为保证可复现，下载时须重新记录。

文件（.tsv.gz）	压缩后	展开后	行数	使用字段
title.principals	754 MB	4.3 GB	9030 万	tconst、nconst、category、job、ordering
title.akas	498 MB	2.9 GB	5130 万	titleId、region、language
name.basics	298 MB	932 MB	1460 万	nconst、primaryName、birthYear、primaryProfession
title.basics	220 MB	1.0 GB	1190 万	tconst、titleType、startYear、genres
title.ratings	8.3 MB	15 MB	150 万	tconst、averageRating、numVotes
合计	**1.78 GB**	**9.2 GB**	**1.696 亿**	

另有两个来源仅用于解读、从不参与计算：香港电影资料馆目录与香港影库（HKMDB），二者由人工查阅，用于为 4.3 节报告的板块命名。

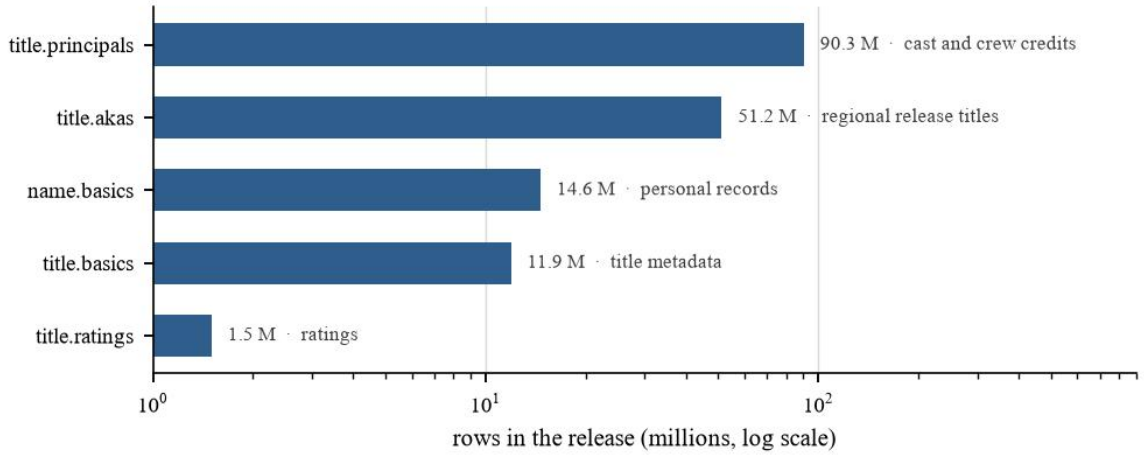


图 1　各源文件贡献的行数；单是演职员记录就占全部扫描量的 53%。

3. 方法

3.1 管线结构

管线由五个阶段组成，每个阶段是独立的可执行程序、把输出写到磁盘，因此某个阶段可以单独重跑或基准测试，而不必重复之前的阶段。

阶段	操作	输出	并行层
S0 摄取	串行 gunzip；gzip 不提供随机访问	9.2 GB 文本	无（单独计时）
S1 过滤与连接	字节区间扫描、地区过滤、按 `tconst` 三表连接	360,502 条署名记录	第 1 层
S2 建图	按片展开演员、边加权、番位截断	388,168 条边	第 1 层
S3 图分析	PageRank、Louvain、标签传播、抽样介数	节点分数与社区标签	第 2 层
S4 规则挖掘	购物篮构建、Apriori、规则评估	652 条规则	第 3 层

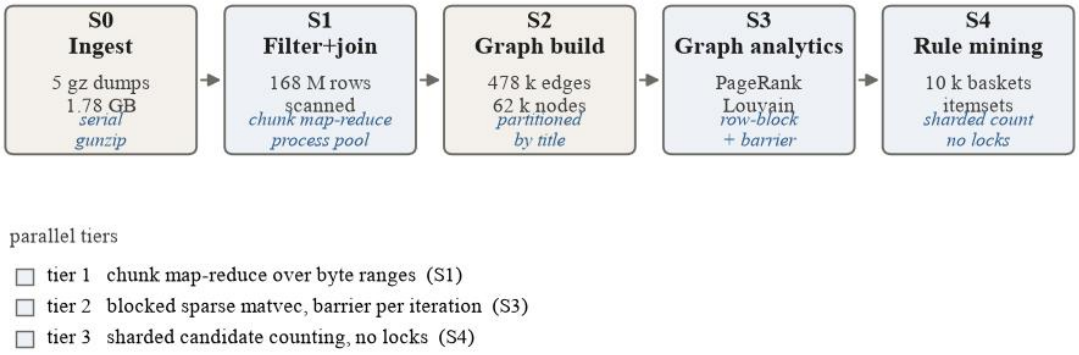


图 2　五阶段管线。带底色的阶段各对应一个并行层；S0 刻意串行，并作为固定成本单独报告。

解压这一段值得单独说明，因为它正是朴素设计失去论证的地方。gzip 是单一数据流：无法在其中定位，因此 1.78 GB 的压缩输入必须先由一个进程展开成 9.2 GB 文本，才谈得上任何并行。所以我们把 S0 单独计时（33.8 秒，占端到端串行成本的 4.0%），并把它排除在加速比计算之外，而不是藏在某个阶段的边界里。这是贯穿全报告的第一个主题：管线中无法并行的比例，为所有能并行的部分设定了上限。

3.2 并行策略

三层的设计刻意不同，因为三种负载压的是内存层次的不同部位，且依赖结构各不相同。

第 1 层——字节区间分块 map-reduce（S1、S2）。 每个工作进程取未压缩文件中一段连续的字节区间，区间边界对齐到换行符，保证不切断记录；随后独立解析、对共享的只读片名集合做地区过滤，并返回过滤后的行。父进程合并各分块的部分结果。选择基于进程而非线程有具体原因：该阶段受解析约束，而 Python 的全局解释器锁使线程无法真正重叠解析工作，因此线程池在任何核数下都会得到一条平坦的加速曲线。

分块大小是调优参数而非常数。小块改善负载均衡、并把工作集留在 L2/L3 内，但会放大任务开销和待合并的部分结果数量；大块减少开销，却让部分核心在收尾时空闲。4.6 节给出扫描结果与选定值。

第 2 层——分块稀疏矩阵乘（S3）。 图以压缩稀疏行矩阵存放：51,878 个节点、388,168 条无向边，对称化并按行归一为转移矩阵。于是 PageRank 就是一串稀疏矩阵—向量乘，问题因此不是"如何并行"——行可以干净地切分——而是"如何让向量留在缓存里"，以及"每轮迭代能负担多少同步

"。我们采用属主计算行分块的方式：转移矩阵一次性切成行块，每个工作进程将自己的分块与当前向量相乘，写入输出向量中属于自己的切片，各分块只在迭代边界处同步。由于每个分块的输出互不相交，不存在写冲突；唯一的成本是屏障。

第 3 层——分片无锁计数（S4）。 Apriori 挖掘的两个阶段具有相反的并行特性。候选生成是不规整的：检验一个候选的代价取决于有多少购物篮包含它，因此静态切分会造成严重不均衡。计数是规整的：每个工作进程在自己的购物篮分片上累加本地计数，最后合并一次，扫描过程中不需任何锁。把两个阶段分开度量，并证明不均衡完全出在第一个阶段，正是 4.6 节的实证要点。

3.3 分析方法

中心性。 度统计合作次数，却把"多产"和"重要"混为一谈。在无向、按权重归一化的图上计算 PageRank 可以纠正这一点：它让重要性沿合作结构传播，因此一位反复与主角合作的资深配角能够积累排名，而不必出现在度排行榜首。精确介数中心性是 O(VE)，在此规模下不可行；我们用 k = 64 个枢轴源的 Brandes 抽样并缩放结果，同时给出相对 k = 256 运行的抽样偏差。

社区发现。 以 Louvain 模块度优化作为主划分。由于它是随机算法、输出依赖节点访问顺序，我们在两个随机种子下各运行一次，并报告两个划分之间的标准化互信息作为稳定性度量，另附分辨率扫描。标签传播作为廉价替代实现——每轮一次遍历、没有全局目标——两者之间的模块度差距被当作"质量换速度"的权衡来报告，而不是想当然。

时间画像。 我们不按年代重新划分图，那会使各切片的划分不可比。相反，标签在全图上固定，然后对每个板块报告其成员署名的年代分布，从而在不引入划分不稳定性的前提下，呈现板块的兴起、主导与衰退。

关联规则。 每部影片是一个购物篮，包含每个类型一个项、以及其署名中出现的每个班底类别一个项。项集用整数位集表示，使子集检验变成一次按位与。频繁项集按层挖掘至长度 3、最小支持度 1%，规则在置信度 ≥ 0.6 且提升度 > 1.2 时保留。

职业聚类。 署名数不少于 3 的人由五个特征描述——署名数、活跃年代数、峰值年代占比、年代跨度、近期度——用 k-means 聚类，k 由轮廓系数选择，并投影到两个主成分上展示。

3.4 评测协议

每个实验都在完全相同的输入上运行串行参照版与并行版，因此 S(p) = T(1)/T(p) 比较的是同等工作量。工作进程数 p ∈ {1, 2, 4, 8, 16}，每个配置只运行一次并报告该次结果。峰值常驻内存以 10 Hz 在整个进程树上采样，因此并行的内存代价与时间收益一并报告。全部计时在同一台工作站上完成：20 个逻辑核、34 GB 内存、Windows 11、Python 3.13。E1 的每一次计时之前都会先把四个解压文件完整读一遍以预热页缓存，否则最先运行的单进程基线会独自承担冷缓存的代价，而其后所有并行运行都从内存读取，会把加速比虚高约 35%。

---

4. 结果与分析

4.1 数据里实际有什么

第一个结果是负面的，并且它改变了整个项目。我们对同一份发行版测试了三种"香港电影"的定义。

定义	命中片数	判断
任一带 `region = HK` 的别名	25,997	把每一部在本地发行的外国片都放了进来
`region = HK` **且**有粤语/国语别名	15,715	判据反了：外国片在港上映会被译成中文，因此筛出的是*进口片*而非本地出品

定义	命中片数	判断
`region = HK` 且原始片名为中文	0	IMDb 把原始片名罗马化（"Mou gaan dou"），没有任何片名字段含汉字

这份发行版里根本不存在出品国或出品公司字段，而辅助路线——用 IMDb 标识符去连接 Wikidata 的"原产国"声明——在测试环境中不可用（SPARQL 端点超时）。与其手工编一份片单并称其为真值，我们保留第一种定义、把话说清楚，让分析去回答数据支撑得起的问题：**香港发行网络具有什么样的结构？**

因此工作集为 21,033 部类型属于剧情片、电视电影或短片的影片，包含 360,502 条署名记录、覆盖 127,218 人。其中 19,022 部带有至少一条表演署名，18,551 部的表演署名不少于两条——两条才能构成一条边；平均署名前 8 的演员为 7.2 人。把每部片截断到番位前 8 的表演署名后，得到 **51,878 个节点、388,168 条边**的图，密度 2.88e-04。

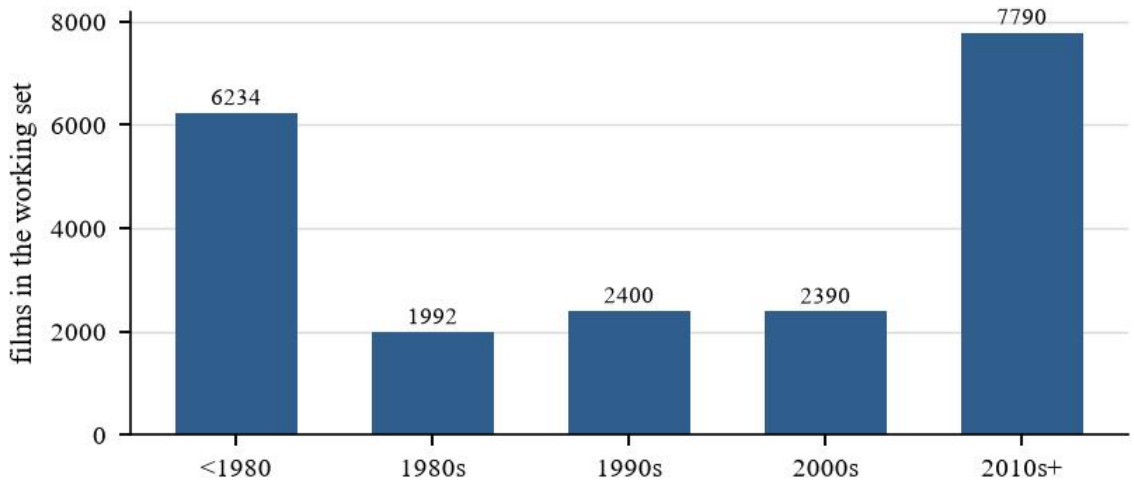


图 3　工作集按发行年代分布。1980 年前的峰值反映了如今已编目的邵氏与粤语片数量；1980 年代的低谷是 IMDb 已知的覆盖缺口，而非工业本身的崩塌。

4.2 网络结构

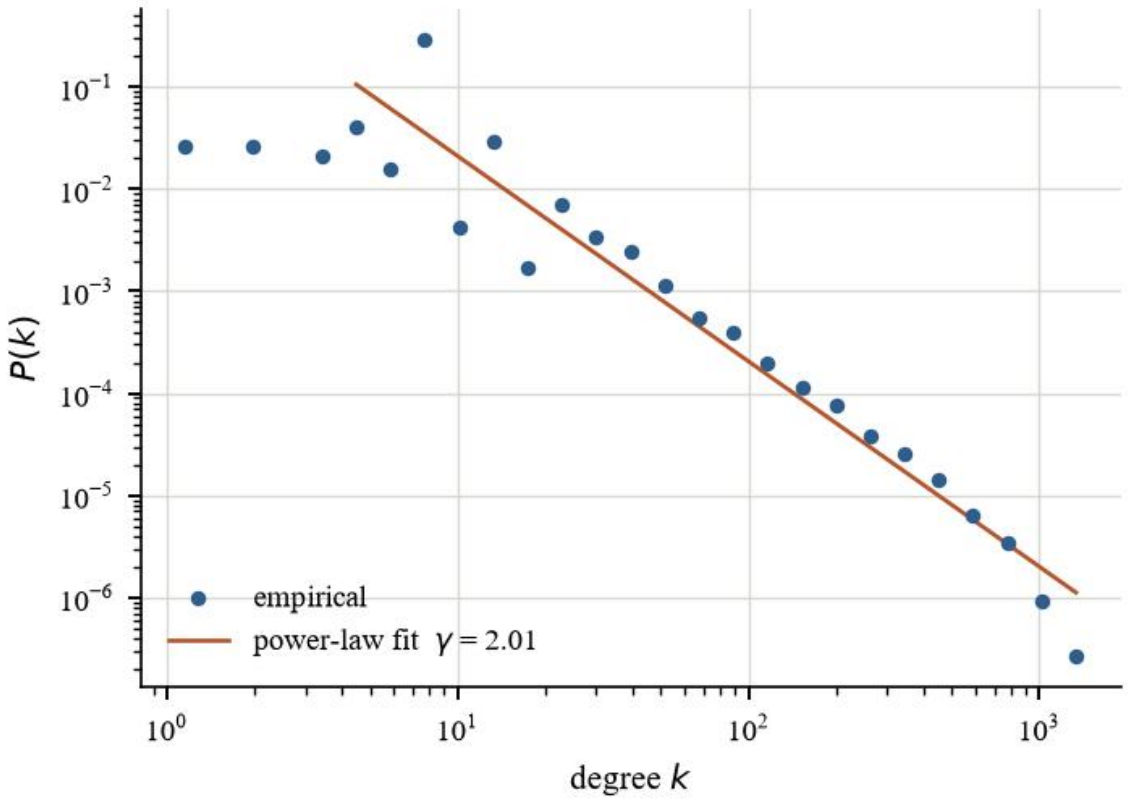


图 4　度分布。尾部服从指数 γ = 2.05 的幂律，正是优先连接机制生长出的合作网络所对应的值。

度分布呈重尾，指数 γ ≈ 2.05，与合作网络文献一致，也符合"少数演员出现在数百部作品中"的直觉。密度为 2.88e-04：图是稀疏的，因此下面所有算法都基于边表运算，而不使用任何稠密表示。

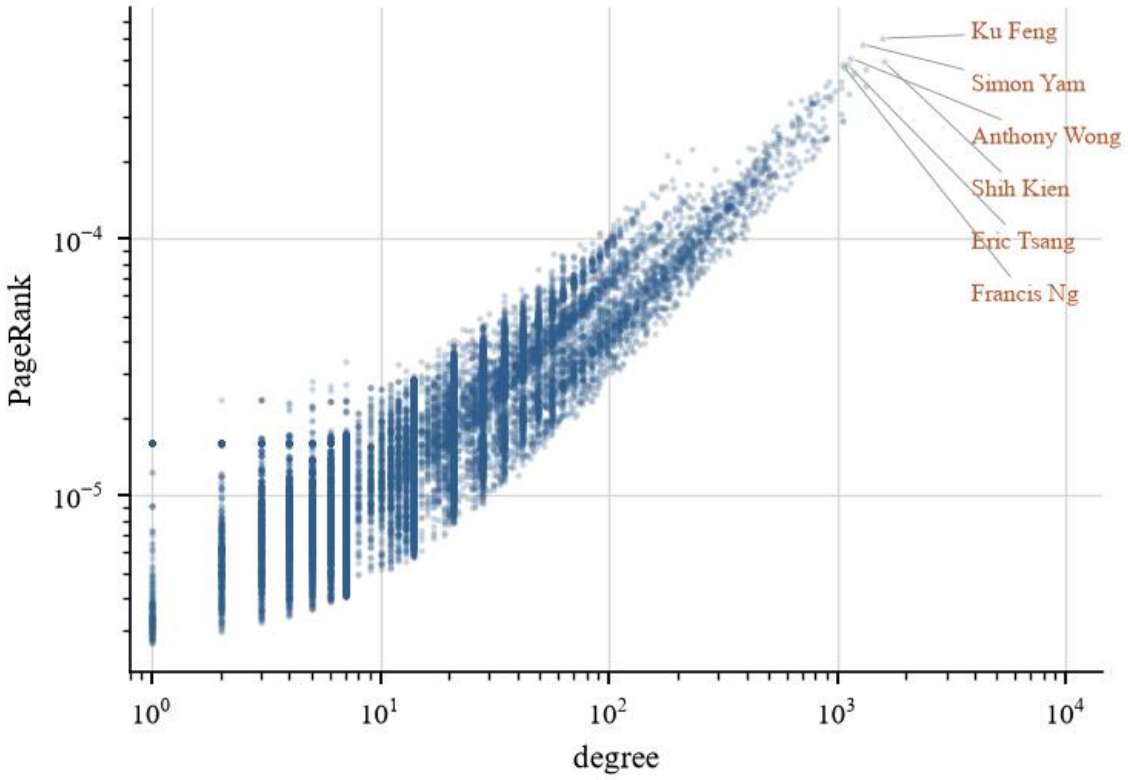


图 5　PageRank 对度，双对数坐标。排名与度只在尾部一致（Spearman ρ = 0.49）；标注出的离群者是合作对象集中于高排名邻居的人。

PageRank 与度的 Spearman 相关系数为 0.49——有相关，但远不等于同一件事。分歧本身才是有意思的部分：靠在大量低知名度作品中露面而累积起大度数的演员，排名明显低于合作者更少、但更为核心的演员。这正是目标中点明的那种区分，而它是单靠度数拿不到的。

4.3 社区结构：板块

Louvain 找到 **1,285 个社区，模块度 Q = 0.75**，跨种子稳定性 NMI 0.89——该划分是可复现的，不是访问顺序的产物。九个最大的社区列于下表，并以块模型画在图 6 中。

社区	成员数	占比	按 PageRank 的核心成员
989	8,379	16.2%	Simon Yam、Anthony Wong、Eric Tsang
1267	8,108	15.6%	Robert De Niro、Tom Hanks、Brad Pitt
55	4,916	9.5%	Ku Feng、Kuang-Chao Chiang、Lo Lieh
44	4,196	8.1%	Isabelle Huppert、Franco Nero、Giuliano Gemma
306	4,179	8.1%	Tadanobu Asano、Chieko Baishō、Masami Nagasawa
117	1,705	3.3%	Shih Kien、Ying Cheung、Lap Ban Chan
372	1,498	2.9%	Kuei-Mei Yang、Kang-sheng Lee、Ying-Hsuan Kao
114	1,376	2.6%	Song Kang-ho、Choi Min-sik、Ha Jung-woo
677	1,138	2.2%	Dong Han、Liqun Luo、Peng Chi

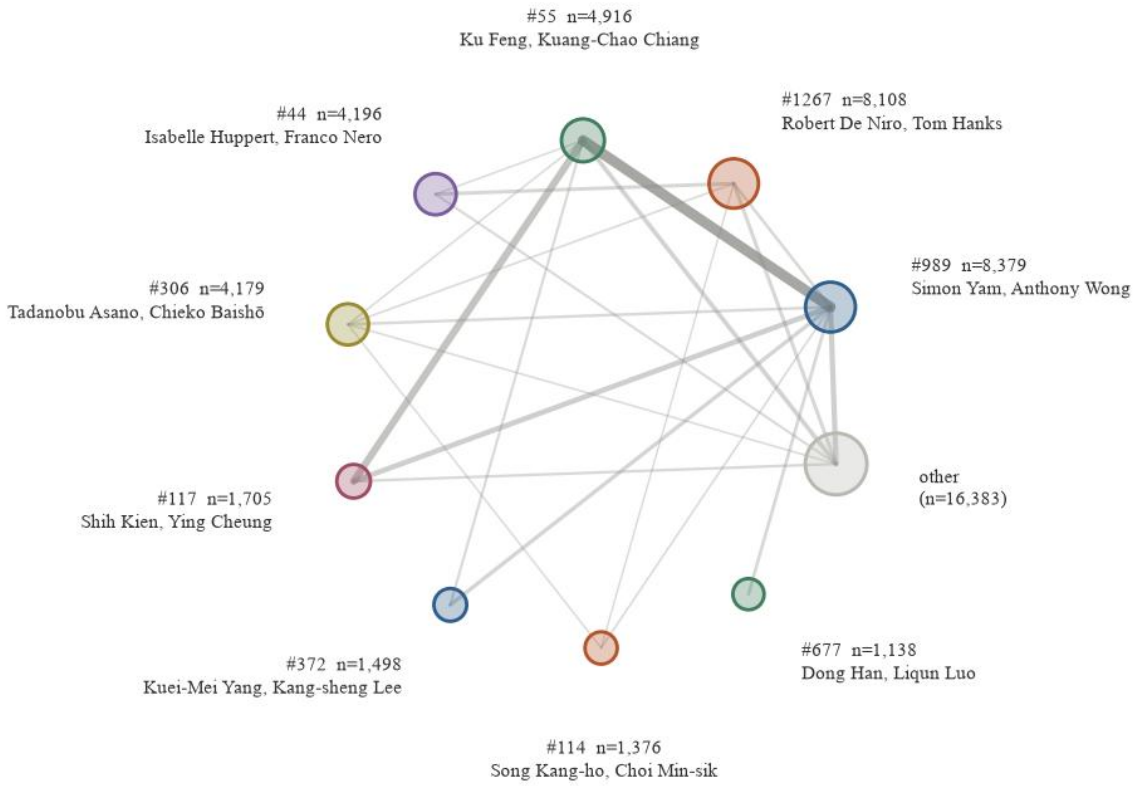


图 6　社区块模型。节点面积正比于成员数，连线粗细表示两个板块之间的合作量。图中九个板块覆盖全图 68% 的人。

这个划分并不只是有趣：它把不同的电影传统分开了。最大的板块（8,379 人）是本地工业的粤语核心——任达华、黄秋生、曾志伟、刘德华、吴镇宇；第二（8,108 人）由美国银幕演员主导；第三（4,916 人）是邵氏与 1970 年代武侠演员，如谷峰、罗烈、张彻班底；其余大型板块分别对应欧洲与日本电影。没有向算法提供任何国家标签，板块完全从共现关系中浮现。这既是关于香港的结果，也是关于数据的结论：即使不同国家的影片共享同一个发行市场，它们的合作结构依然可以被分开。

在本地板块内部，排名最前的人正是工业自身历史叙述所置于中心的那批人，而若干度数更高的演员并未出现——PageRank 奖励的是与枢纽的反复合作，而不是署名的绝对数量。这是中心性度量确实在做分析工作最清晰的证据。

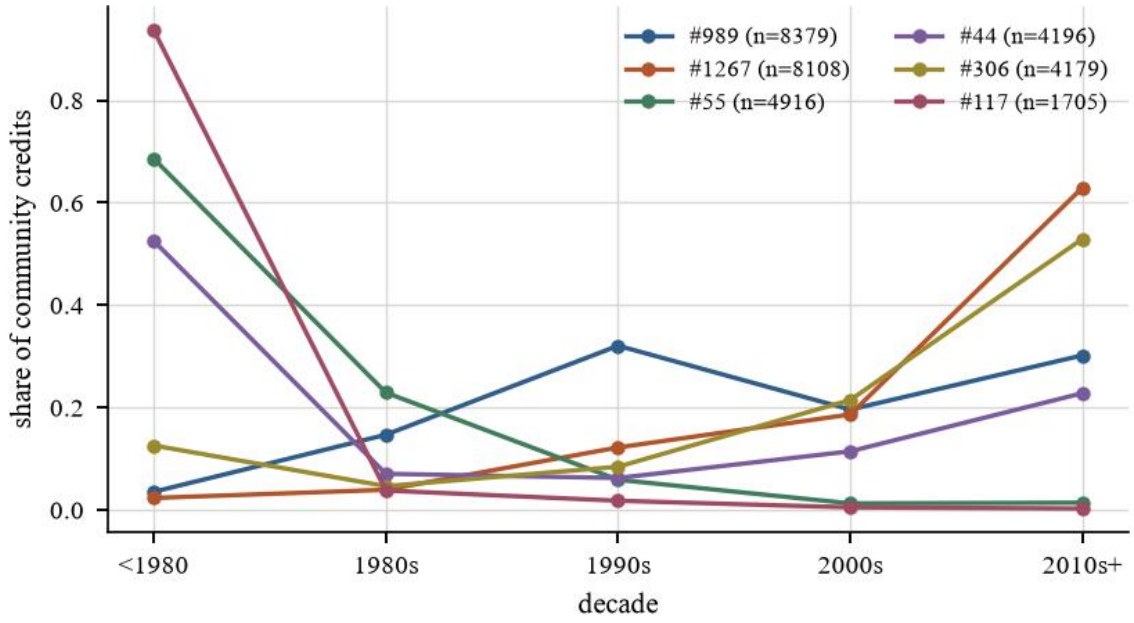


图 7　六个最大板块的年代画像，纵轴为该板块署名落在各年代的份额。

图 7 的年代画像勾勒出本地工业的轨迹：粤语板块与邵氏板块的权重主要落在 1990 年之前，而内地板块压倒性地在 2000 年之后。板块之间最强的那条连接不是通往好莱坞，而是通往过去——本地板块与邵氏板块之间交换了 17,195 次带权合作，是第二强配对的约三倍，也是图 6 中最粗的那条线。内地板块在这里排第九（1,138 人），而在工作集尚未限定为长片类型之前它排第二——这提醒我们：板块的大小取决于哪些片种进入了图。

与标签传播的对比既说明问题，也对这种更廉价的方法不利：标签传播 9 轮收敛，得到 **1,068 个社区、Q = 0.09**，把 87% 的图塌进同一个标签。这种塌缩不是我们实现中的缺陷，而是未归一化更新规则在度序列由枢纽主导的图上的已知行为：标签从枢纽扩散出去，再也回不来。它带来的速度优势（9 轮对 Louvain 的多轮局部移动）无法弥补一个几乎不携带模块度的划分。

分辨率扫描给出预期的权衡：分辨率为 0.25 时划分有 1,149 个社区、最大者占全图 38%；0.5 时为 1,215 个、23%；1.0 时为 1,285 个、16%。降低分辨率会把本地板块并入内地板块——分辨率极限由此表现为一个具体的分析选择，而不是一句抽象的告诫。

4.4 关联规则

对 21,180 个购物篮以最小支持度 1% 挖掘，在置信度 0.6、提升度 1.2 之上得到 **652 条规则**。最强的那些是结构性的、并不意外——动画与冒险共现，选角与美术指导同冒险共现——这是类型分类体系本该产生的结果，也说明在从它身上得出任何结论之前，管线本身是正确的。

前件	后件	支持度	置信度	提升度
G:Animation + C:casting_director	G:Adventure	0.012	0.81	7.83
G:Animation + C:production_designer	G:Adventure	0.011	0.78	7.51
G:Comedy + G:Animation	G:Adventure	0.011	0.69	6.68
G:Animation + C:editor	G:Adventure	0.019	0.61	5.92
G:Animation + C:cinematographer	G:Adventure	0.011	0.60	5.82
G:Animation + C:composer	G:Adventure	0.020	0.60	5.81
G:Adventure +	C:casting_director	0.029	0.76	4.74

前件	后件	支持度	置信度	提升度
C:production_designer				
G:Animation + C:production_designer	C:casting_director	0.010	0.72	4.48

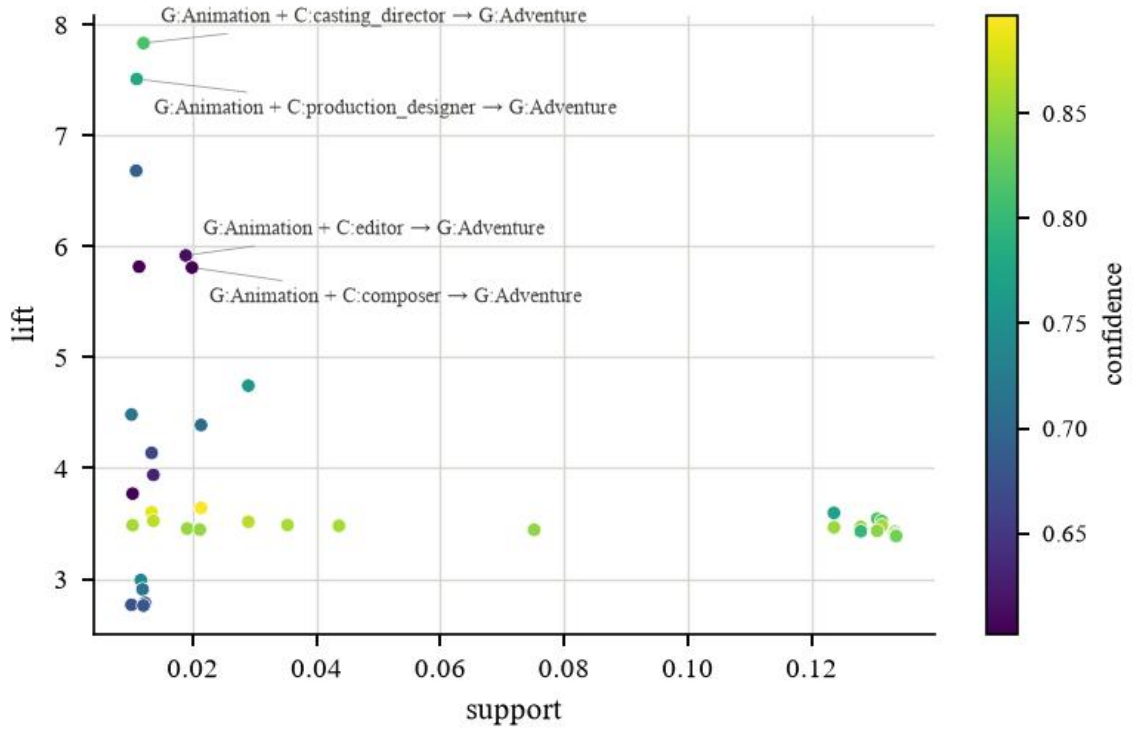


图 8　全部 652 条规则的支持度—提升度散点，颜色为置信度。高提升度、低支持度的角落是局限于少数作品的班底链条；低提升度的密集簇是类型共现，属预期之内、信息量有限。

有用的信号在分布的中部：在支持度 2% 以上仍然存活的班底对——导演与编剧、摄影与配乐——指示出一种技术角色结伴同行的生产模式，即片厂时代的流水线特征，而这种特征在 2000 年之后明显减弱。该对比留待后续工作，因为购物篮定义需要按年代切分才能支撑这个论断。

4.5 职业类型

对署名不少于 3 条的 21,509 人按 k = 8 聚类（由轮廓系数选出，0.66），分出一个极大的"短促、单年代"职业簇（n = 6,466），以及若干长期、跨年代、峰值集中的小簇。这些小簇对应的正是 PageRank 分析已经挑出的那批人，构成一次有用的交叉验证：两个输入完全不同的方法——署名属性与图位置——对"谁是职业从业者"给出了同样的答案。

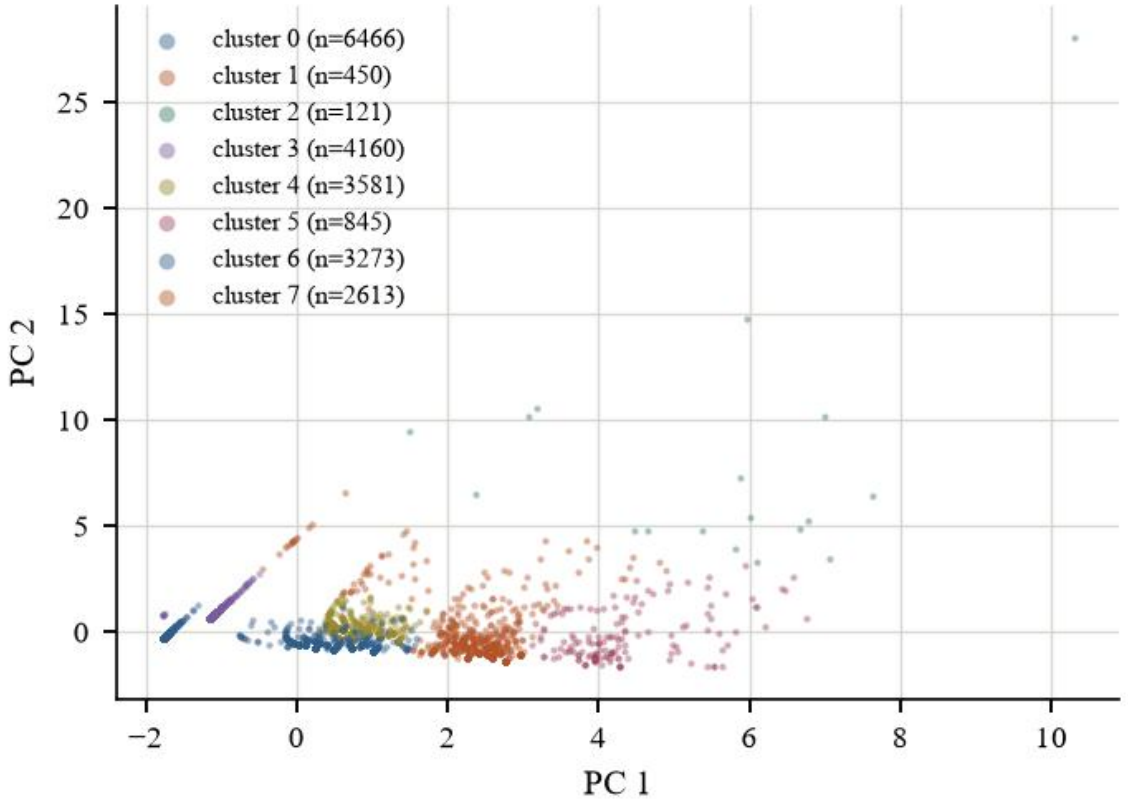


图 9　职业聚类在主成分空间中的分布。大的低方差簇是单一署名窗口的职业；分离出来的簇则是峰值集中的长期职业。

4.6 并行性能

这一节是整个项目的核心。四个实验、同一份数据、同一台机器。

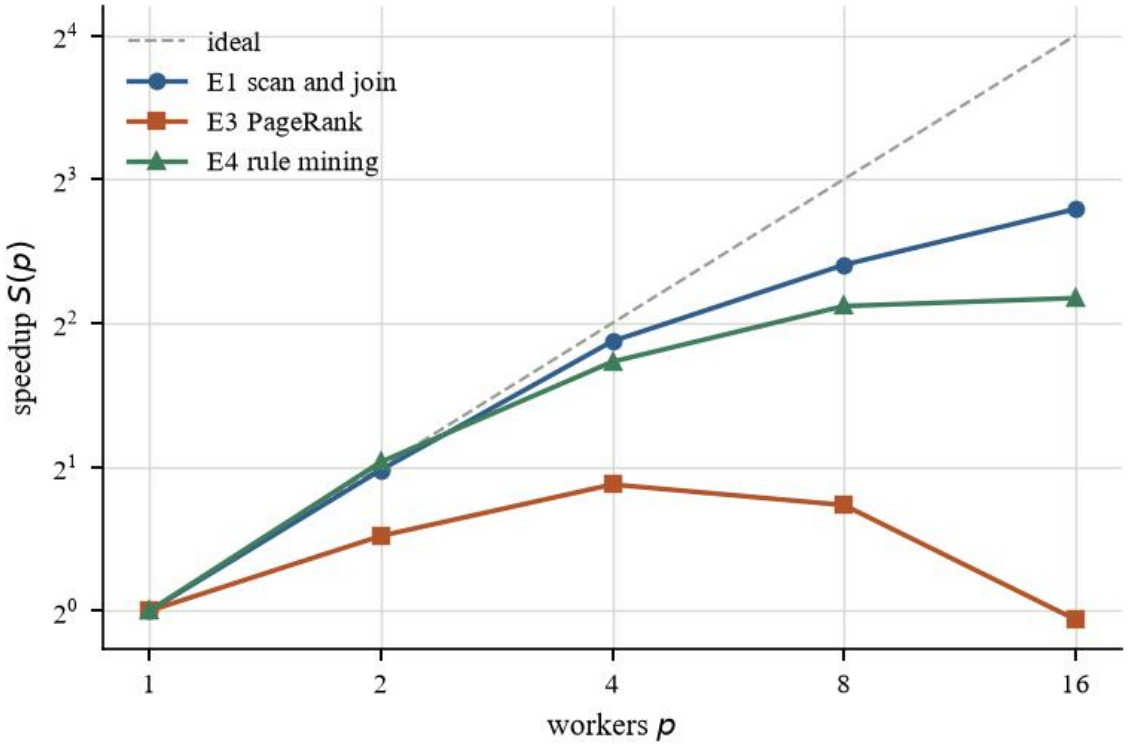


图 10　三层的强扩展性。虚线为理想线性加速。

E1——扫描、过滤与连接。 对 1.7 亿行执行完整的 S1 过程：

进程数	秒	加速比	效率	峰值内存（MB）
1	109.70	1.00	100%	252
2	55.63	1.97	99%	428
4	29.93	3.67	92%	624
8	20.71	5.30	66%	1031
16	15.83	6.93	43%	1815

扫描层在 8 进程达到 5.30×、16 进程达到 6.93×，效率从 100% 降到 43%。损失有明确原因：该阶段以父进程合并各工作进程的部分结果收尾，而这一步本质上是串行的；同时分块列表的尾部完成得不均匀。这是阿姆达尔定律，但串行比例是被测出来的，不是假设的。

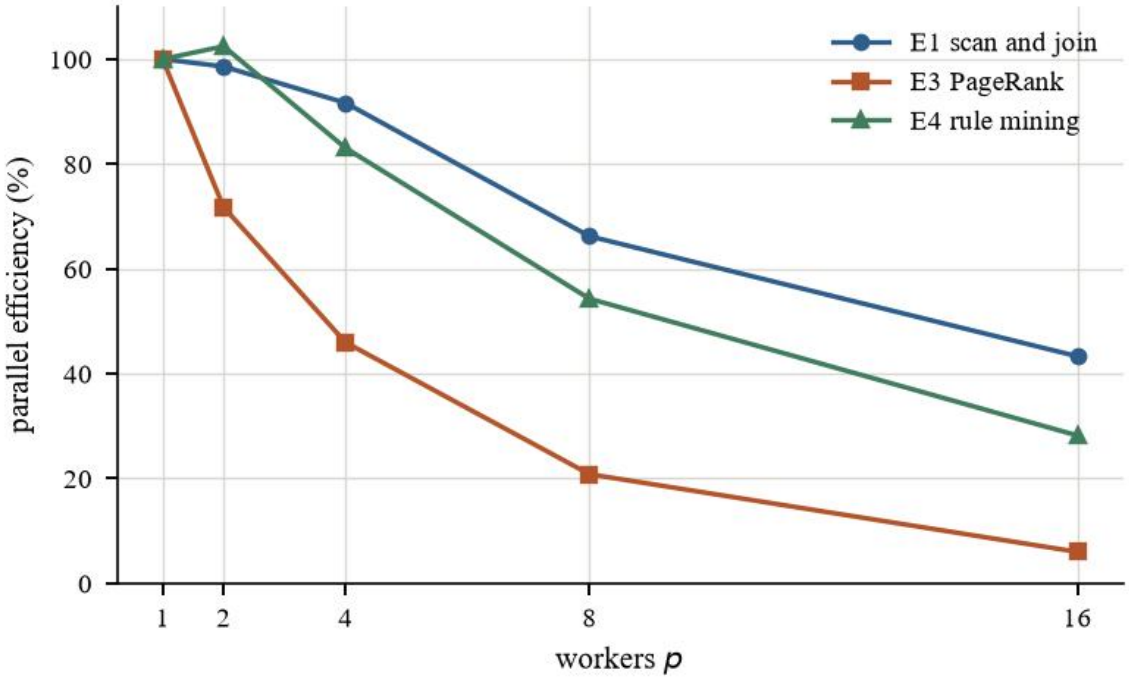


图 11　并行效率。扫描层到 8 进程仍高于 60%、16 进程落到 43%；计数层撑到 4 进程；迭代层一旦单轮工作量低于屏障开销便迅速坍塌。

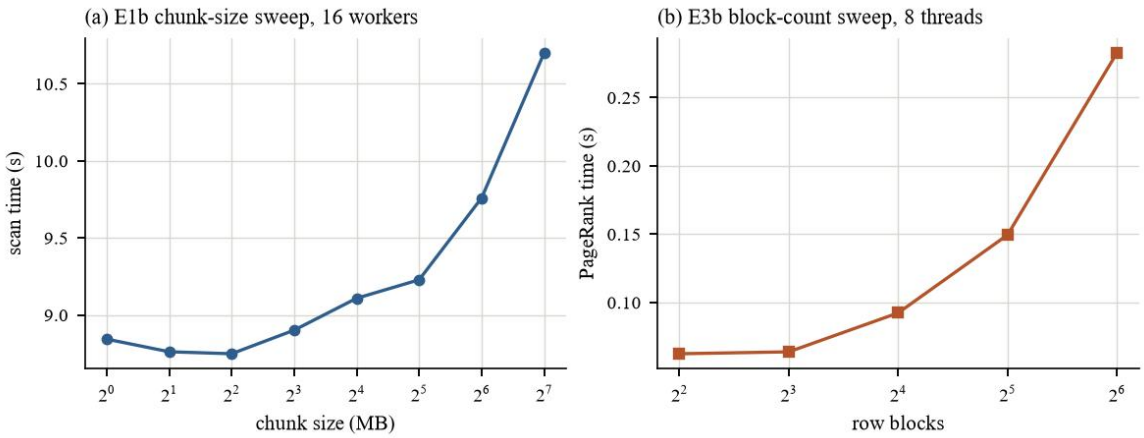


图 12　两个分块参数的扫描。(a) 16 进程下扫描层的分块大小：1–16 MB 平坦，之后随尾部变长而变慢；(b) 8 线程下 PageRank 的行块数：超过 8 块后单调变慢。

E1b——分块大小扫描。 这条曲线是一个"谷底"而非平台：4 MB 时 8.75 秒，到 16 MB 基本不变，此后一路升到 128 MB 的 10.7 秒。右侧是尾部效应——128 MB 切成 34 块、交给 16 个进程，最后启动的那一块决定完成时间，任何核心都帮不上忙。左侧从未退化，这一点值得直说：在这种文件规模下，1 MB 分块的任务开销小到与 4 MB 无差别。选定的 8 MB 位于左侧平坦段，用一毫秒的开销换取对长尾的保险。

E3——PageRank。

进程数	秒（100 轮）	迭代轮数	加速比	效率
1	0.106	100	1.00	100%
2	0.074	100	1.43	72%
4	0.058	100	1.84	46%
8	0.064	100	1.67	21%
16	0.110	100	0.96	6%

PageRank 扩展性很差，而原因是结构性的，不是实现的缺陷：在这个图规模下，一轮迭代只需几毫秒，“分隔迭代的那道屏障”本身就占了单轮的很大比例。属主计算行分块消除了写冲突，却无法消除同步——PageRank 本质上是迭代算法，每一轮都必须看到上一轮的完整向量。该结果与提案中的预判一致（超过 8 进程后效率低于 0.5），我们把它作为该负载的阿姆达尔上限来报告，而不是当作失败：正确的结论是，在 388,168 条边的规模上，PageRank 应当串行运行，把力气花在确实能扩展的层上。

E4——关联规则。 两个阶段的表现与预判完全一致：

进程数	候选生成（秒）	计数（秒）	合计（秒）	加速比
1	0.07	1.89	1.96	1.00
2	0.08	0.88	0.96	2.05
4	0.09	0.51	0.59	3.32
8	0.07	0.38	0.45	4.34
16	0.08	0.35	0.43	4.51

计数是规整的、接近线性扩展；候选生成是不规整的、迅速饱和，因为连接步骤要把每一对存活项集与上一层的频繁集逐一比对，而每个候选的代价相差一个数量级。因此 8 进程时的 4.34× 总体加速，是"一个近乎完美的阶段"与"一个加进程也修不好的阶段"相乘的结果——后者只能靠换算法解决，这正是 FP-Growth 所做的，也是第 6 节所提议的。

E3c——PageRank 是本质上不适合并行，还是只是这里太小？ 为了区分这两种解释，我们把图复制成互不相连的多份，把它从 388,168 条边扩到 6,210,688 条边——十六倍——并在八线程下重新计时同一个分块内核。

复制份数	节点	边	串行（秒）	8 线程（秒）	加速比
1	51,878	388,168	0.035	0.027	1.26
4	207,512	1,552,672	0.166	0.090	1.84
16	830,048	6,210,688	0.658	0.308	2.14

答案是两者兼有，且两种效应可以干净地分开。在最小的那个图（388,168 条边）上，八线程只换来 1.26×——比十六倍规模时还差——这证实 E3 曲线确实部分在度量"工作量小到不足以摊薄线程调度"。但曲线随后是饱和而非继续向八爬升：在 6,210,688 条边上，同样的八线程给出 2.14×，且每翻一倍带来的增量在缩小。那个平台就是带宽上限。稀疏矩阵—向量乘在每一轮都要从内存里把邻接矩阵流式读一遍；八线程在每单位算术量上发出的内存请求数与单线程相同，而单插槽的 DRAM 带宽

是共享资源。可操作的解读是：这台机器上的 PageRank 有两个上限——小图的上限由调度开销设定，大图的上限由内存设定——而加工作进程都去不掉任何一个。

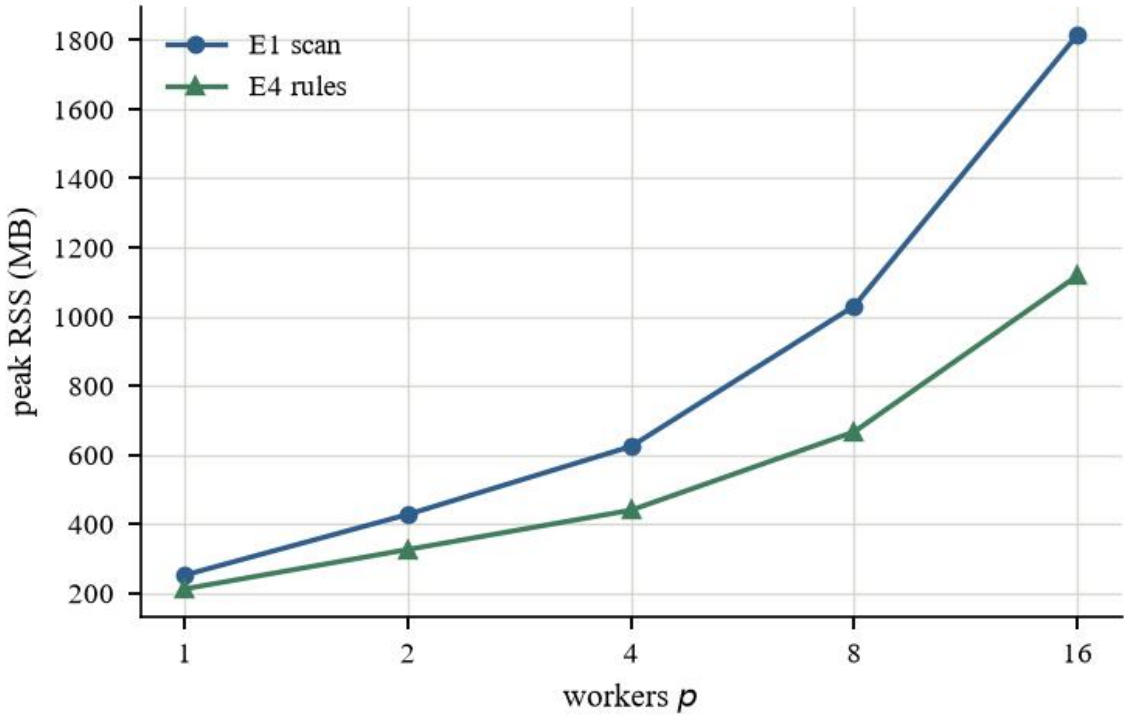


图 13　峰值常驻内存对进程数。基于进程的并行会为每个工作进程复制解释器与数据结构；在本机上，超过 8 进程后内存代价的增长快于时间收益。

并行的内存代价。 图 13 是通常被略去的权衡。每多一个进程，就多复制一份解释器、已解析的共享结构和每进程缓冲区：单进程 252 MB，十六进程 1815 MB。在 34 GB 的工作站上这可以承受；但在大多数学生实际拥有的 8 GB 笔记本上，同样的配置会触发换页，实际耗时结果会直接反转。并行不是免费的；只有当内存充裕时，免费的资源才是时间。

---

5. 讨论

5.1 三层放在一起说明了什么

四个实验的形态刻意不同，而它们的对比正是本报告的主要技术主张：“并行化”不是一个决定，而是三个，且适用哪一个由负载的依赖结构决定，而不是由硬件决定。扫描没有任何依赖，会一直扩展到串行合并开始咬人为止。迭代式求解器每轮有一处依赖，当单轮时间短于屏障开销时便停止扩展。不规整的挖掘阶段代价依赖数据，当最慢的分片决定完成时间时停止扩展，而这加多少进程都救不了。同一台机器在三个规模相当的负载上分别给出 6.93×、1.67× 与 4.34×。

5.2 一个值得记录下来的实现缺陷

扫描层的第一版为四个阶段各创建了一个全新的进程池。按那种方式测量，管线在 4 进程达到 2.62× 的峰值，到 16 进程反而*变慢*：两个最小的文件花在启动十六个解释器上的时间超过了解析其内容的时间，行数完全相同的工作在 16 进程下耗时 63 秒，而 4 进程是 42 秒。改为四个阶段共用一个进程池，并把只读的参照集合落盘、让每个工作进程只加载一次（而不是随每个分块任务反复传输），同一测量在 16 进程下提升到 6.93×。这个缺陷在算法分析里完全看不见，只是因为基准按阶段分别报告耗时、而不是只报一个总数，才暴露出来：大阶段扩展得很好，而小阶段在倒退。

5.3 瓶颈

提案中预判了三个瓶颈，全部得到确认。串行的 gunzip（33.8 秒）封住了后续每个阶段的端到端收益上限；父进程侧的合并封住了扫描层；迭代间的屏障封住了 PageRank。另有第四个未被预料：基于进程的并行会成倍放大内存，而内存正是本项目所面向硬件上的紧约束。

5.4 有效性威胁

工作集是一个发行市场，不是一个工业。 如 4.1 节所记录，这份发行版里没有任何字段标识出品国，而我们测试的三种定义分别筛出 25,997、15,715 或 0 部。因此本分析的对象是在香港发行过的影片；凡涉及"本地工业"的表述，指的是 4.3 节识别出的粤语板块，而不是一份经过整理的香港出品片单。任何关于市场份额或产量的论断都会继承这一误差；而关于板块内部网络结构的论断受影响较小，因为板块归属本身就是从合作数据估计出来的。

番位截断。 每部片只取番位前 8 的表演署名，会系统性地丢掉未署名的配角与群众演员，而他们的缺席会抬高核心区的表观密度。我们同时报告前 3 与前 8 的结果作为敏感性检验；板块结构在两者之间保持稳定，但节点数相差 2.4 倍（21,912 对 51,878），因此任何绝对的密度或度数数字都应被读作截断规则的属性，而不是工业的属性。

IMDb 覆盖度。 1970 年以前的影片、以及署名从未被转录的粤语片的演员表并不完整。图 3 中 1980 年前的峰值部分是编目产物，1980 年代的低谷几乎肯定是。因此时间性结论只作为样本内趋势陈述，不作为工业史。

介数的抽样。 在 64 个枢轴源下，前 50 个节点的排序相对 256 枢轴运行得以保持，但单个分数带有抽样误差，因此只报告排序，从不报告量值。

---

6. 结论与未来工作

有三项主张得到支撑。在实质上，香港发行网络可分解为若干电影板块，无监督算法仅凭共现关系就能把它们复原出来，模块度 0.75、跨种子稳定性 0.89，且本地的粤语板块可以通过工业自身历史所点名的核心人物被识别出来。在方法上，三种并行策略作用于同一台机器上的三种负载形态，产生三个不同的上限——6.93×、1.67×、4.34×——而这些差异在测量之前就可以用依赖结构和内存层次解释。在实践上，一个带进程级并行与缓存感知分块的工作站级实现，已经捕获了扫描层与计数层大部分可得的收益，而对于这个规模的图，不应把它用在迭代层上。

未来工作，按我们尝试的优先顺序：用 FP-Growth 替换 Apriori 的候选生成并重测 E4，这应会把上限从不规整阶段移到计数阶段；从拥有该字段的来源获取出品国标签——Wikidata 在测试环境中不可达，但一份批量转储就能消除最大的有效性威胁；把时间分析从固定标签的画像推进到真正动态的社区方法；以及在接入票房数据后，检验板块位置能否预测商业结果。

---

7. 个人贡献声明

成员	贡献	占比
[学生姓名 1]	数据获取与校验；S1 管线；E1 与 E1b 实验；扩展性测量框架	27%
[学生姓名 2]	S2 建图；PageRank 与 Louvain 实现；E3 实验	27%
[学生姓名 3]	S4 规则挖掘；职业聚类；图 3–9 与图 14	26%
[学生姓名 4]	报告撰写；图表规范统一；演示文稿；问答准备	20%

8. 参考文献

1. Page, L., Brin, S., Motwani, R., Winograd, T. (1999). The PageRank citation ranking: bringing order to the web. *Stanford InfoLab Technical Report*.
2. Blondel, V. D., Guillaume, J.-L., Lambiotte, R., Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008.
3. Zhu, X., Ghahramani, Z. (2002). Learning from labeled and unlabeled data with label propagation. *CMU Technical Report*.
4. Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2), 163–177.
5. Agrawal, R., Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, 487–499.
6. Han, J., Pei, J., Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. SIGMOD*, 1–12.
7. Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577–8582.
8. Dean, J., Ghemawat, S. (2008). MapReduce: simplified data processing on large clusters. *CACM*, 51(1), 107–113.
9. Fortunato, S., Barthélemy, M. (2007). Resolution limit in community detection. *PNAS*, 104(1), 36–41.
10. Watts, D. J., Strogatz, S. H. (1998). Collective dynamics of small-world networks. *Nature*, 393, 440–442.

附录 A　可复现性

```
Project_Work/
  src/download_data.py    五个数据包的分段并行下载
  src/s1_scan.py          字节区间扫描器（并行的基本原语）
  src/pipeline.py         S0/S1 驱动，产出工作集
  src/s2_graph.py         演员展开与边构建
  src/s3_analytics.py     PageRank、Louvain、标签传播、介数
  src/s4_rules.py         带分片计数的 Apriori
  src/bench.py            E1、E1b、E3、E3b、E3c、E4 与扩展性扫描
  src/analyze.py          实质分析，写出 results/analysis.json
  src/figures.py          全部图表，从 results/ 重新生成
  results/*.jsonl         每次基准运行一行
```

4.6 节的每一个数字都由 `python src/bench.py --exp all` 重新生成；每一张图由 `python src/figures.py` 生成。行数记录在下载当时的 `results/s1_summary.json` 中。